AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 1

Workflows or Agents: Building Effective Agents

A team built an autonomous agent to file expense reports. It took 40 seconds per report, and it sometimes skipped the receipt check. The steps never changed. Four plain function calls and two model calls would have done the job faster, cheaper and the same way every time.

Module	AI06 of 13
Lesson	1 of 10
Prerequisites	AI03 L1–L3, AI03 L6
Exam relevance	**Direct.** TS 1.6 (task decomposition), and it frames all of Domain 1 (27%)
Reading time	~40 minutes

Contents

1. Workflows or Agents

Lesson 1 of 10 · Agents and MCP

Workflows or Agents

Key idea

In a workflow, your code decides the path. In an agent, the model decides the path. That one choice sets your cost, latency, predictability and failure modes. **Use the simplest thing that works: a single call first, then a workflow when the steps are known, and an agent only when the steps can't be known in advance.**

> *Before: the Messages API, structured output, context engineering. Teaches: Anthropic's five workflow patterns, when an agent earns its cost, and how to decompose a task. Used by: every lesson in AI06, AI07, AI08 (3.*

The hook: an agent that didn't need to be one

Expense filing has the same steps every time: extract the fields, validate them, check the receipt against the policy, submit. Because the team had built it as an agent, the model chose the order at run time. It usually chose well, but sometimes it submitted before checking.

Measure	As an agent	As a workflow
Who decides the steps	The model, on each run	Your code, always the same
Can a step be skipped?	Yes, occasionally	No
Cost per report	Varies	Fixed and known
How you debug it	Read a transcript	Read the code and the failing step

Anthropic's Building Effective Agents opens with the same advice: "we recommend finding the simplest solution possible, and only increasing complexity when needed." For many applications, it says, "optimizing single LLM calls with retrieval and in-context examples is usually enough."

Intuition: a recipe or a chef

A **workflow** is a recipe. Anyone following it gets the same result, and if the soup is salty, you know which step to change. An **agent** is a chef handed a basket of ingredients: "make dinner." You get flexibility, and a chef can handle a missing ingredient. But you'll wait longer, it costs more, and sometimes dinner isn't what you pictured.

If you're making the same soup a thousand times, you want the recipe.

Mechanism: the ladder of complexity

Anthropic's definitions are precise, and worth learning exactly:

→ **Workflows** are "systems where LLMs and tools are orchestrated through predefined code paths."
→ **Agents** are systems where LLMs "dynamically direct their own processes and tool usage."

Both are built on the **augmented LLM**: "an LLM enhanced with augmentations such as retrieval, tools, and memory."

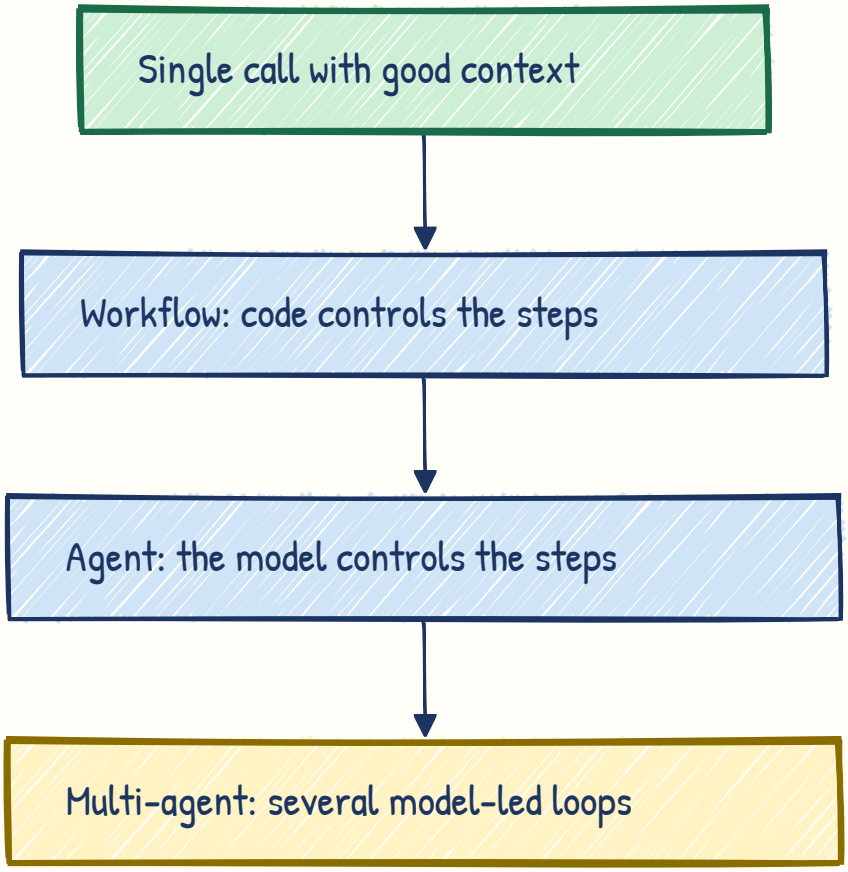


Figure 1. The ladder. Each rung adds capability, cost and new ways to fail, so climb one only when the rung below has demonstrably failed.

The five workflow patterns

Pattern	How it works	Use it when	Example
Prompt chaining	Each call handles the last one's output, with optional code gates	The task splits cleanly into fixed steps	Outline, check the outline, then write
Routing	Classify the input, then send it to a specialized path	Categories are distinct and better handled apart	Billing vs login tickets
Parallelization	Run calls at once: **sectioning** (different parts) or **voting** (same task, many tries)	Independent parts, or wanting several views	Separate safety and content checks
Orchestrator-workers	A model breaks down the task, delegates, and synthesizes	Subtasks can't be predicted in advance	Multi-file code changes
Evaluator-optimizer	One model generates, another critiques, in a loop	Criteria are clear, and iterating measurably helps	Translation with critique

The key line between parallelization and orchestrator-workers: with orchestrator-workers, "subtasks aren't pre-defined, but determined by the orchestrator based on the specific input."

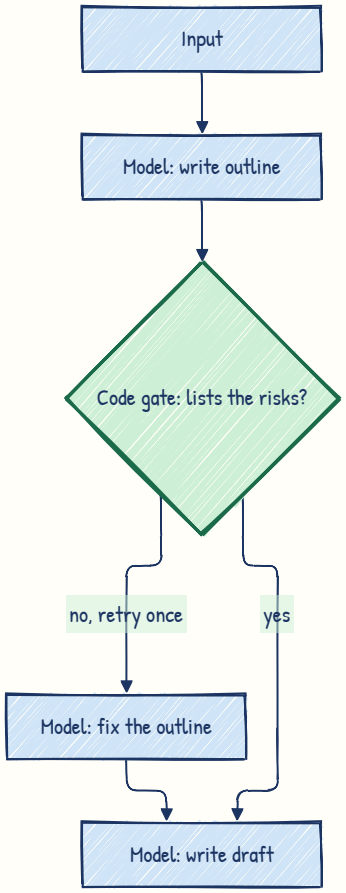


Figure 2. Prompt chaining with a gate. Code checks each output before the next step runs, so a bad early result stops rather than spreading.

The chain's main goal, in Anthropic's words, is "to trade off latency for higher accuracy, by making each LLM call an easier task."

When an agent earns its place

Agents suit open-ended problems where the number of steps can't be predicted or hardcoded, in trusted environments. Check four questions, and if any answer is no, stay lower on the ladder:

Check	Question	Example of "no"
Complexity	Is it hard to specify the steps in advance?	"Extract the invoice total"
Value	Does the outcome justify the cost and latency?	A $0.02 autocomplete suggestion
Viability	Can Claude do this kind of task well?	No tools exist for the needed system
Cost of error	Can mistakes be caught and undone?	Irreversible payments with no review

Anthropic's Appendix adds what good agent uses share: conversation combined with action, clear success criteria, feedback loops, and human oversight. It names customer support and coding agents.

Errors compound

Each step of an agent can go wrong, and later steps build on earlier ones.

Chance each step is right	Steps	Chance all are right
99%	20	**82%**
95%	10	**60%**
90%	10	**35%**

These figures assume independent steps, which is a simplification. Real agents can notice an error and recover, and that's the very thing feedback loops buy. But the arithmetic shows why you verify along the way, rather than only at the end.

Decomposition: chain or adapt?

This is what **TS 1.6** tests.

Situation	Choose	Why
Reviewing a pull request across known aspects	**Prompt chain**: each file, then a cross-file pass	Predictable, and it avoids attention dilution
"Add thorough tests to this legacy codebase"	**Dynamic**: map the structure, find high-impact areas, then plan and adapt	You can't know the work until you've looked
Extracting 12 fields from invoices	**Single call** with a schema	No steps to decompose
"Why did checkout errors rise last night?"	**Dynamic**	Each finding decides the next question

Why the obvious fixes fall short

Tempting move	Why it disappoints	Better
"Use an agent, because it's flexible"	You pay for flexibility you don't use	A workflow with code-controlled steps
Tell the agent "always check the receipt"	A prompt lowers the failure rate, but it isn't zero (TS 1.4)	Make the check a code step
Add a framework first	Abstraction can hide prompts, and make debugging harder	Start with the API directly
One giant prompt with every step	Attention dilutes, and you can't tell what failed	Chain it, with gates
Multi-agent from the start	About 15× chat's token use, per Anthropic (AI06 L6)	Earn it with an eval

How Anthropic approaches it

Principle (from the post)	What it means in practice
Simplicity	Fewer moving parts, and add them only when "it demonstrably improves outcomes"
Transparency	Show the agent's planning steps
Agent-computer interface	Treat tool docs and testing like a user interface (AI06 L3)

On frameworks, the post is balanced. They simplify calls and tool plumbing, but they "often create extra layers of abstraction that can obscure the underlying prompts". So it suggests starting with LLM APIs directly, and understanding any framework's code. AI07 takes this up.

The post also notes that the tooling landscape has moved on since it was written in December 2024, and it points to Managed Agents documentation for Anthropic's current approach. The patterns themselves remain useful.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: one agent for all requests. It's easy to start, and every request pays agent prices.

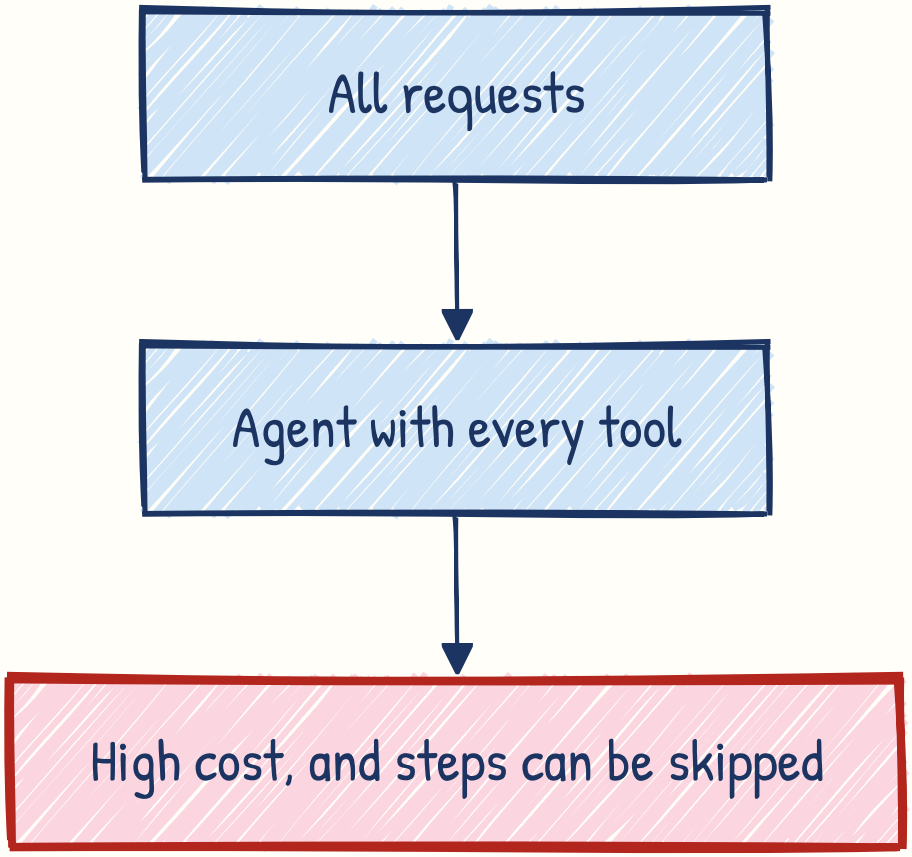


Figure 3. Baseline. Every request runs the full agent, even those with a known set of steps.

Improved: fixed workflows for everything. It's predictable and cheap. New problem: **novel requests fail, because no path exists for them.**

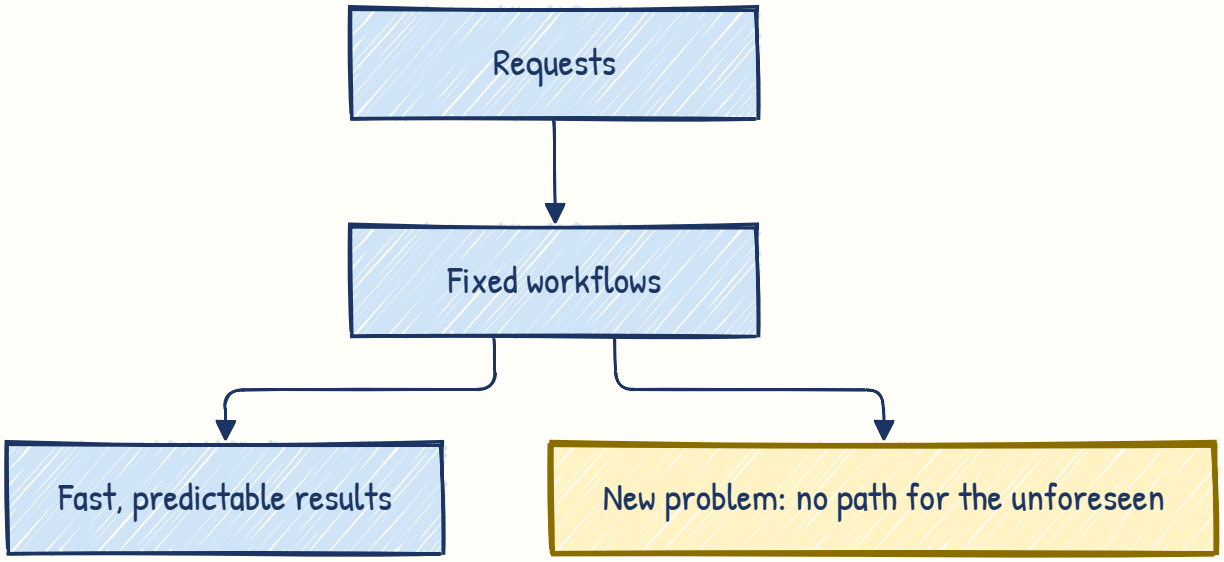


Figure 4. Improved. Workflows are predictable, but anything unforeseen drops out with no path forward.

Production: route to the lowest rung that works.

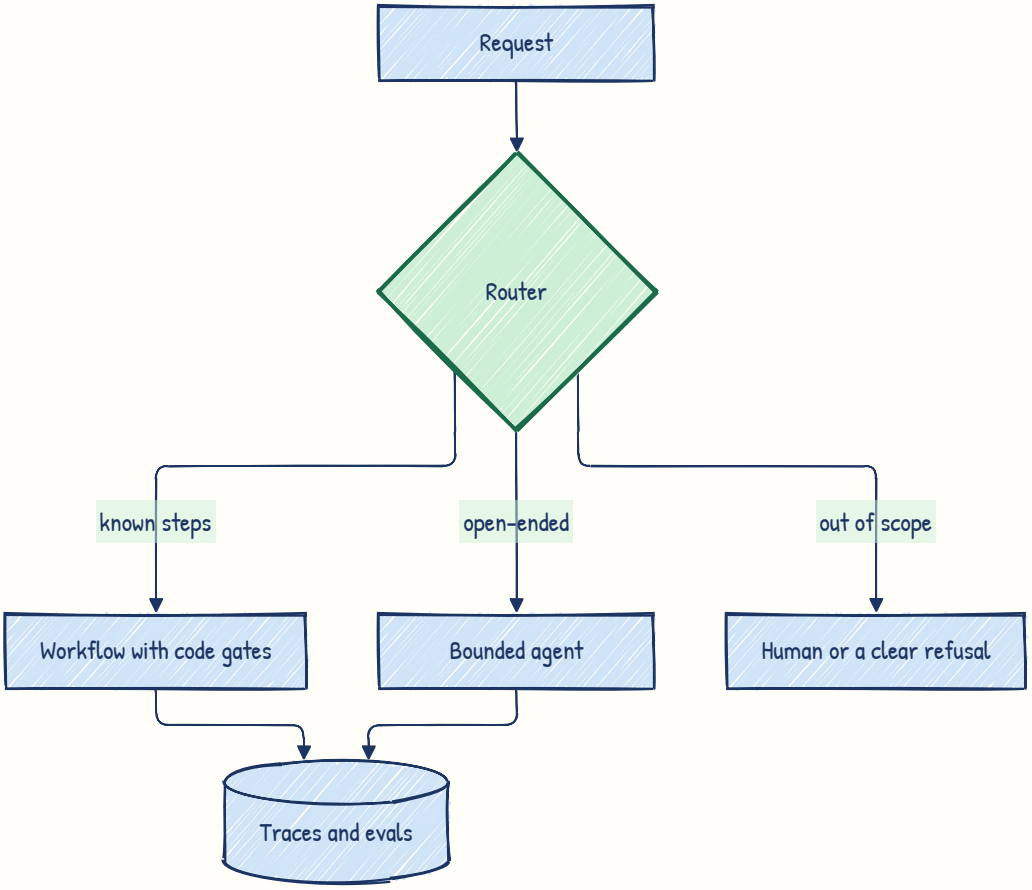


Figure 5. Production. A router sends known request types to workflows, and only open-ended ones to a bounded agent. Both report into the same evaluation logs.

Component	Why it exists	How it communicates
Router	Keeps known work cheap	Rules, or a small model
Workflow	Guarantees the order of steps	Plain code calling the model
Bounded agent	Handles the unforeseen	Tool loop, with limits (AI06 L2)
Traces and evals	Show which path earns its cost	Async

Failures

Failure	Containment
Router misclassifies	Evaluate it separately, and let workflows hand off to the agent
Gate fails repeatedly	Stop and report. Don't let bad output go further
Agent exceeds its budget	Limits, then a partial answer (AI06 L2)
A workflow step's model call fails	Retry the step alone, not the whole chain

Java

From `code/ai06-agents/.../Workflows.java`. The "model" is a scripted function, so this tests control flow and nothing else.

```
String output = steps.get(i).apply(current);
String problem = gate.apply(output);
if (problem != null) {
    trace.add("step " + (i + 1) + " rejected: " + problem);
    output = steps.get(i).apply(current + " [fix: " + problem + "]");
```

The decision that matters: **the gate is code**, and it passes the specific problem back. Real output:

```
step 1 rejected: outline must list risks
step 1 ok: Outline: scope, risks, rollout
step 2 ok: Draft from <Outline: scope, risks, rollout>
billing workflow   <- I was charged twice
account workflow   <- Login link expired
general agent      <- My parcel arrived damaged and the invoice looks wrong
```

Notice the last line: anything the rules don't recognize goes to the agent, and that's the design working as intended.

Trade-offs

Decision	Gain	Cost
Workflow	Predictable, cheap, debuggable	Rigid
Agent	Handles the unknown	Cost, latency, variance
Router in front	Cheapest path per request	Another failure point
Evaluator-optimizer	Better results where criteria are clear	More calls, and it may not converge
Voting	More confidence	Multiplies cost

When not to use an agent: the steps are known, a mistake can't be undone or caught, or the budget per request is tiny.

Common mistakes

1. **Starting with an agent.** Try a single call first.
2. **Calling every LLM app an agent.** If code picks the steps, it's a workflow.
3. **Using a prompt for a must-happen step.** Make it code.
4. **Using a fixed plan for an unknown problem.** Investigation has to adapt.
5. **Using dynamic planning for a fixed review.** That adds cost and variance for nothing.
6. **Adopting a framework before understanding the loop.** Debugging suffers.

What to say in an interview

"I'd climb the ladder only as needed: a single call, then a workflow, then an agent. Agents earn their cost when steps can't be known in advance and errors can be caught. For a pull request review I'd chain per-file passes and a cross-file pass, whereas for an open-ended investigation I'd let the plan adapt to what's found. Any must-happen step becomes code."

Certification lens

How this lesson maps to CCAR-F

TS 1.6: fixed chains for predictable work, dynamic decomposition for open-ended investigation.

Also **TS 1.1's** distinction between model-driven decisions and pre-configured decision trees.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Code Generation with Claude Code · Medium. A team needs to review pull requests of about 30 files for the same four concerns. One combined pass misses issues in later files. What's the best decomposition?

A. A chain: analyze each file separately, then run one cross-file integration pass.

B. A dynamic agent that decides at run time which files seem worth reading.

C. One pass with a larger context window.

D. Three reviewers per pull request, keeping only issues all three report.

A **A** (TS 1.6). The work is predictable, so a chain fits. B adds variance and can skip files. C adds capacity, but it doesn't fix attention. D discards real issues that only one pass finds.

Q Q2 · Exam style, scenario: Developer Productivity with the Agent SDK · Medium. The request is "add thorough tests to this legacy service", with no documentation available. What's the best first approach?

A. List every file, and write a test per file in alphabetical order.

B. Generate one complete test plan up front, and follow it exactly.

C. Map the structure, identify high-impact areas, then create a prioritized plan that adapts as dependencies appear.

D. Ask the model for a test for the whole service in a single prompt.

A **C** (TS 1.6). Open-ended work needs adaptive decomposition. A ignores impact. B can't anticipate what's discovered. D overloads one pass.

Q Q3 · Exam style, scenario: Customer Support Resolution Agent · Medium. Password resets always follow the same three steps and must never skip identity verification. They currently run through the general agent. What should you do?

A. Add "always verify identity first" to the system prompt in capital letters.

B. Add more few-shot examples of correct resets.

C. Lower the model's effort for reset requests.

D. Route resets to a code-controlled workflow, keeping the agent for open-ended issues.

A **D** (TS 1.4, TS 1.6). Known steps with a must-happen rule belong in code. A and B only lower the failure rate. C doesn't address ordering.

Q Q4 · Calculation · Medium. An agent's steps are each 97% reliable and independent, and tasks take 15 steps. What fraction fully succeed with no error? What per-step rate would give 90%?

A 0.97^15 is about **63%**. For 90% over 15 steps, you need 0.90^(1/15), or about **99.3%** per step. This assumes no recovery, and a real agent's ability to recover is why feedback loops matter.

Q Q5 · Best decision · Hard. Match a pattern to each: (a) a legal summary must be checked against 5 criteria, repeating until it passes; (b) a chat message needs safety screening while it's also being answered; (c) a change request touches an unknown set of files.

A (a) Evaluator-optimizer, since criteria are clear and iterating helps. (b) Parallelization by sectioning, which Anthropic's post uses as an example. (c) Orchestrator-workers, because the subtasks depend on the input.

Revision - Lesson 1

→ Workflow: code picks the path. Agent: the model picks the path.

→ Climb only as needed: single call, workflow, agent, multi-agent.

→ Five patterns: chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer.

→ Orchestrator-workers differs from parallelization: subtasks are decided at run time.

→ Agent checks: complexity, value, viability, cost of error.

→ Errors compound: 95% per step over 10 steps is 60%.

→ Fixed chain for predictable work, dynamic plans for open-ended work.

→ Must-happen steps belong in code, not prompts.

→ Anthropic's principles: simplicity, transparency, a well-built tool interface.

→ Start with the API directly, and understand any framework you adopt.

Your notes